COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE 010- Data Structures and Algorithms**	**SECTION:**	**CPE21S4**
DATE PERFORMED:	8/8/26	**DATE SUBMITTED:**	8/8/26
NAME:	Rayven Ferry C. Licas	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 **[**Queues**]**

1. PROCEDURE OUTPUT

What procedures did you perform?
3. PROCEDURE
ILO A: Create queue using C++ STL
Definition from the CPP reference documentation for the queue STL template. queues are a type of container adaptor, specifically designed to operate in a FIFO context (first-in first-out), where elements are inserted into one end of the container and extracted from the other. Queues are implemented as container adaptors, which are classes that use an encapsulated object of a specific container class as its underlying container, providing a specific
set of member functions to access its elements. Elements are pushed into the "back" of the specific container and popped from its "front".
The underlying container may be one of the standard container class templates or some other specifically designed
container class. This underlying container shall support at least the following operations:

- empty
- size
- front
- back
- push_back
- pop_front

The standard container classes deque and list fulfill these requirements. By default, if no container class is specified
for a particular queue class instantiation, the standard container deque is used

Function	Description
Constructor	Creates and initializes an empty queue object.
Empty()	Checks whether the queue contains any elements. Returns true if empty, otherwise false.
Size()	Returns the number of elements currently stored in the queue.
Front()	Returns a reference to the element at the front of the queue (the next element to be removed).
Back()	Returns a reference to the element at the rear of the queue (the most recently added element).
Push()	Inserts a new element at the rear of the queue.
Emplace()	Constructs a new element directly at the rear of the queue without creating a temporary object.
Pop()	Removes the element at the front of the queue.
Swap()	Exchanges the contents of one queue with another queue.

Sample code for Queue STL in C++:

```cpp
// C++ code to illustrate Queue in the Standard Template Library (STL)

#include <iostream>
#include <queue>

void display(std::queue<int> q)
{
    std::queue<int> c = q;

    while (!c.empty())
    {
        std::cout << " " << c.front();
        c.pop();
    }

    std::cout << "\n";
}

int main()
{
    std::queue<int> a;

    // Insert elements into the queue
    a.push(10);
    a.push(20);
    a.push(30);

    std::cout << "The queue a is :";
    display(a);

    std::cout << "a.empty() : " << a.empty() << "\n";
    std::cout << "a.size()  : " << a.size() << "\n";
    std::cout << "a.front() : " << a.front() << "\n";
    std::cout << "a.back()  : " << a.back() << "\n";

    std::cout << "After a.pop() : ";
    a.pop();
    display(a);

    a.push(40);

    std::cout << "The queue a is : ";
    display(a);

    return 0;
}
```

Task: Create C++ code using Queue STL that will pass an array of students' names to the queue. Make observations on the operations performed and show screenshots of code and output console per operation.

B.1. Linked List Implementation

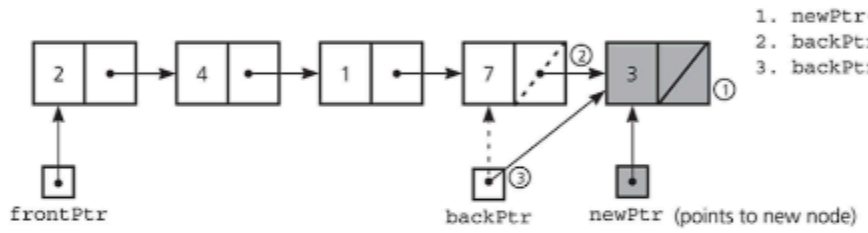
For this section, the linked list implementation will focus on the given behavior of operations as they operate on the nodes and link them together. The nodes are defined the same way as previously, with two compartments: data and pointer to the next.

Operations:

- Inserting an item into a non-empty queue
- Inserting an item into an empty queue
- Deleting an item from a queue of more than one item
- Deleting an item from a queue with one item

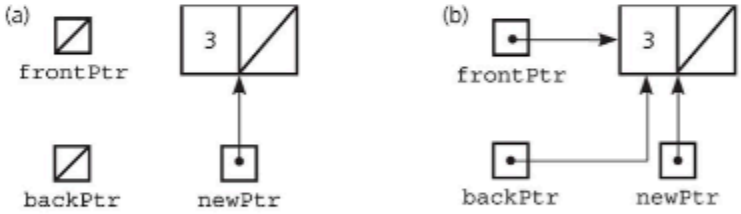
For each item, you are provided an algorithm for the operations. You must implement C++ code for each operation. Then, show each operation and its output as shown through the console.

Inserting an item into a nonempty queue



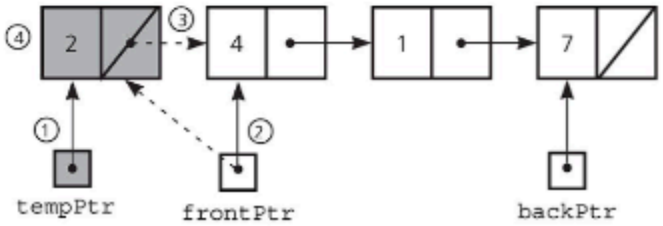

```
1. newPtr->next = NULL;
2. backPtr->next = newPtr;
3. backPtr = newPtr;
```

Inserting an item into an empty queue



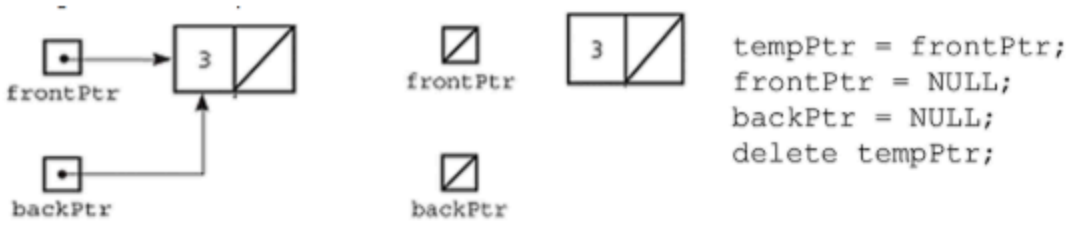

```
frontPtr = newPtr;
backPtr = newPtr;
```

Deleting an item from a queue of more than one item

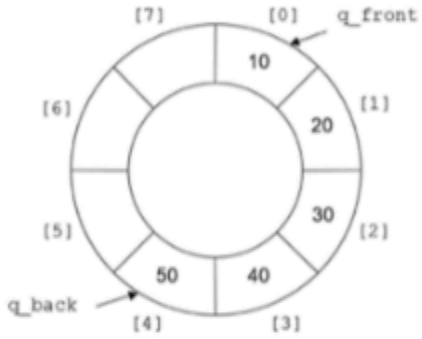



```
1. tempPtr = frontPtr;
2. frontPtr = frontPtr->next;
3. tempPtr->next = NULL;
4. delete tempPtr;
```

Deleting an item from a queue with one item



B.2. Array-Based Implementation



In C++, we could implement an array-based queue as a class. To conserve space, we'll implement it as a "circular queue", an array in which the last position is logically connected back to the first position to make a circle. This is sometimes also called a "ring buffer".

Data members

- **q_array** - Queue array pointer. A pointer to the data type of the items stored in the queue; points to the first element of a dynamically-allocated array.
- **q_capacity** - Queue capacity. The number of elements in the queue array.
- **q_size** - Queue size. The number of items currently stored in the queue.
- **q_front** - Queue front. The subscript of the front (or head) item in the queue.
- **q_back** - Queue back. The subscript of the back (or rear or tail) item in the queue.

Member Functions

The example functions described here correspond to the interface of the queue class in the C++ standard library.

Operation	Description
Queue() *(Constructor)*	Constructs an empty queue.
Empty()	Tests whether the queue is empty. Returns true if the queue contains no elements; otherwise returns false.
Size()	Returns the number of elements currently stored in the queue.
Clear()	Resets the queue size to 0 and restores q_front and q_back to their initial

	values. Does not deallocate dynamic memory or change the queue capacity.
Front()	Returns the element at the front of the queue (the next element to be removed).
Back()	Returns the element at the back of the queue (the most recently added element).
Enqueue()	Inserts a new element at the back of the queue.
Dequeue()	Removes the element at the front of the queue.
Copy Constructor	Creates a new queue as a copy of an existing queue. Unlike vectors or array-based stacks, queue elements may wrap around the array, so the entire queue array must be copied.
Copy Assignment Operator	Replaces the contents of one queue with a copy of another queue. Since queue elements may not occupy consecutive positions, the entire queue array must be copied.
Destructor	Deletes the dynamically allocated queue array and releases its memory.

Member Functions (modified from STL) [bold texts are added modifications]

Your Tasks:

- **Create a queue class that will contain the array.**
- **Operations and member functions described in this section must be in your developed code.**
- **Test the functions and show the output.**

Attribution for 8.2.: http://faculty.cs.niu.edu/~mcmahon/CS241/Notes/Data_Structures/array_based_queue.html

4. SUPPLEMENTARY ACTIVITY

ILO C: Solve problems using queue implementation

Problem Title: Shared Printer Simulation using Queues
Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs to store all the pending jobs somewhere so that it can take them up once its current task is done.

Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation.
You may NOT use the STL.

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).
2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.
3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```
#ifndef QUEUELL_H
#define QUEUELL_H
#include <iostream>

template <typename T>
class qNode {
public:
    T data;
    qNod *next;
};

//creating the new node
template <typename T>
qNode<T> *new_node(T newDATA){
    qNode<T> *newNode = new qNode<T>;
    newNode->data = newDATA;
    newNode->next = NULL;
    return newNode;
}

template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
   //create a node node;
   qNode<T>* newNode = new_node(newData);

   //inserting into an empty queue
   //how do we know that the queue is empty?
   if((*frontPtr)== nullptr && (*backPtr)== nullptr){
      //assign the frontPtr to the newNode
      (*frontPtr)= newNode;
      //assign the backPtr to the newNode
      (*backPtr)= newNode;
      return;
   }
    //point the back next to the newNode:
    (*backPtr)->next = newNode;
    // the newNode will be the new backPtr'
    (*backPtr) = newNode;
```

```
    return;
}

template <typename T>
void dequeue(qNode<T> **frontPtr, qNode<T> **backPtr){
    //how to deteermine if the queue is empty?
    if((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        std::cout<< "The queue is empty. Nothing to dequeue.\n";
        return;
    }

    //creating temporary pointer:
    qNode<T>* tempPtr = nullptr;
    //store the frontptr to the temptr;
    tempPtr = (*frontPtr);

    //check if queue has only 1 node left
    if ((*frontPtr) == (*backPtr)){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete tempPtr;
        return;
    }

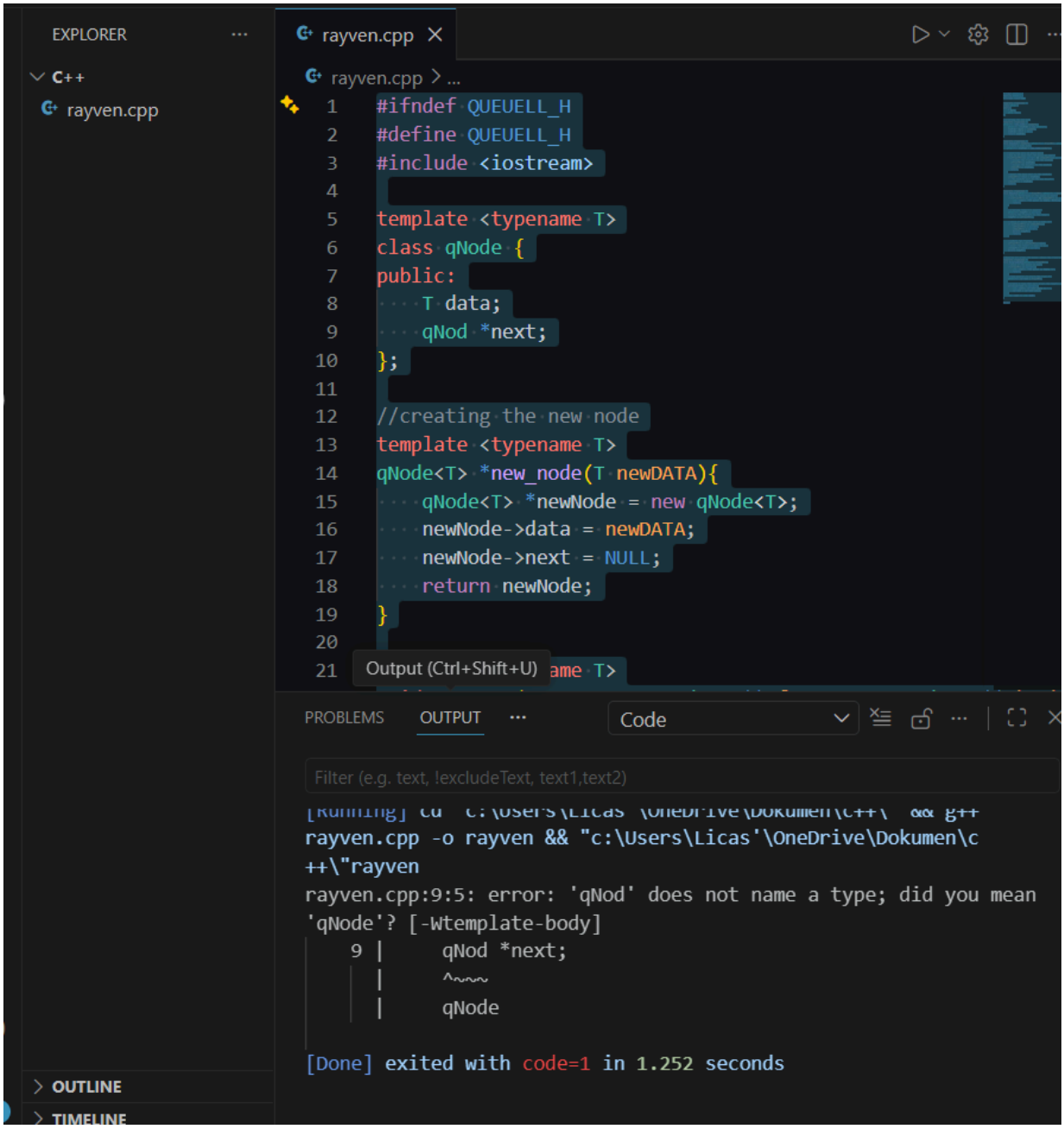
    //update rge frontptr
    (*frontPtr) = (*frontPtr)->next;
    //point the temptr -> next to null
    (*tempPtr)->next = nullptr;
    delete tempPtr;
}

//display all items the queue: check the traverse in singlyLL
template <typename T>
void displayQueue(qNode<T> *frontPtr){
    //check if the queue is empty
    if(frontPtr == nullptr){
        std::cout<< "The queue is empty.\n";
        return;
```

```
    }
    //temp pointer to store the front Ptr
    qNode<T>* current = frontPtr;

    //create a loop that will print the tempPtr->data
    while(current != nullptr){
        std::cout<< current->data << " ";
        current = current->next;
    }
    std::cout << std::endl;
}

#endif
```




```
#include <iostream>
#include "queueLL.h"
#include "queueArr.h"

int main() {
```

```
    qNode<int>* front = nullptr;
    qNode<int>* back = nullptr;

    enqueue(1, &front, &back);
    std::cout<< "front: " << front->data <<" back: "<< back->data<<"\n";
    enqueue(2, &front, &back);
    enqueue(3, &front, &back);
    std::cout<< "front: " << front->data <<" back: "<< back->data<<"\n";

    displayQueue(front);
    dequeue(&front, &back);
    std::cout<< "front: " << front->data <<" back: "<< back->data<<"\n";
    dequeue(&front, &back);
    std::cout<< "front: " << front->data <<" back: "<< back->data<<"\n";
    dequeue(&front, &back);
    dequeue(&front, &back);
    displayQueue(front);

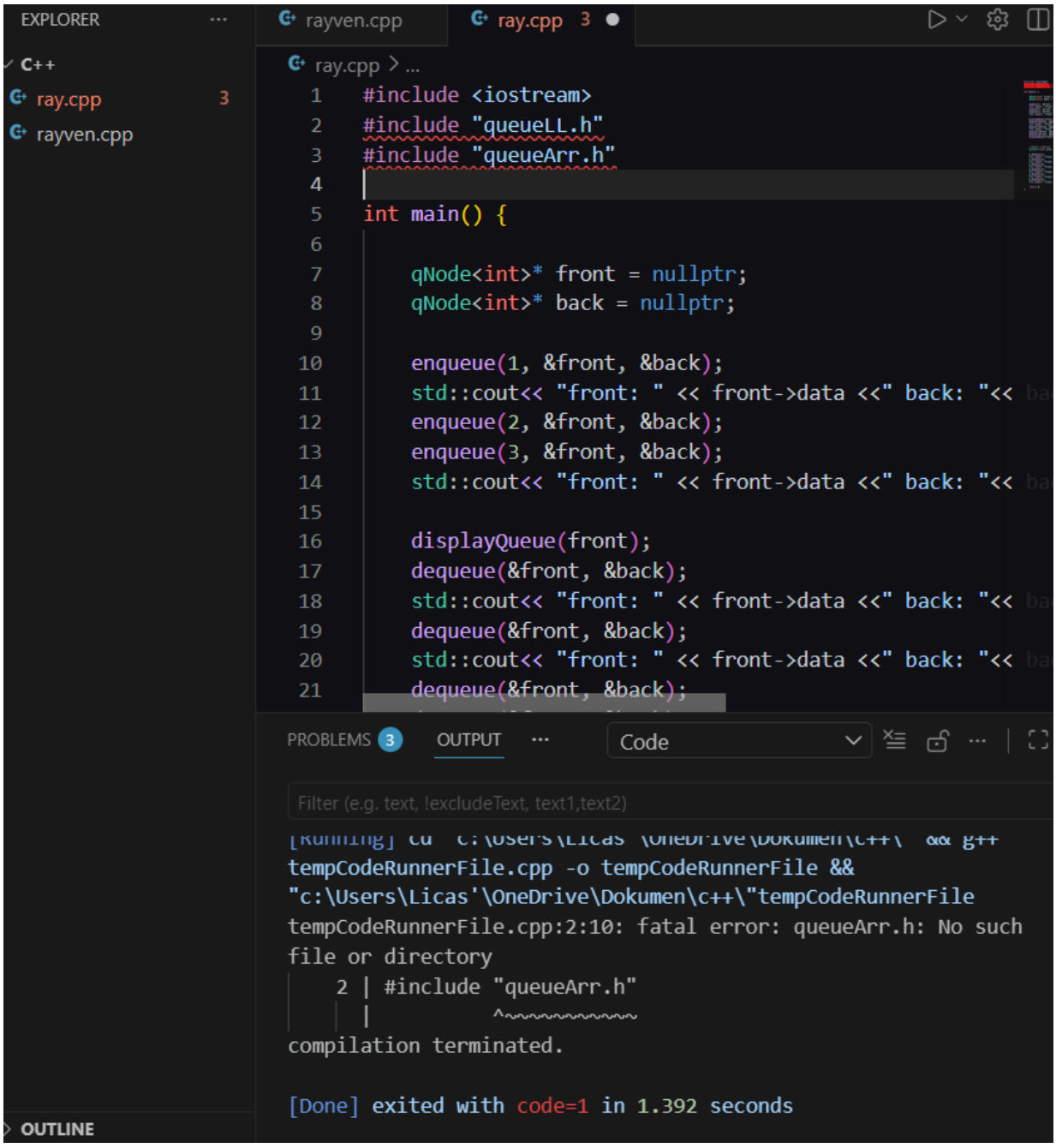


    //object creation
    queueArr<int> q(5);

    q.Enqueue(1);
    std::cout<< "front: " << q.front() <<" back: "<< q.back()<< " size:
" << q.Size() << "\n";
    q.Enqueue(2);
    std::cout<< "front: " << q.front() <<" back: "<< q.back()<< " size:
" << q.Size() << "\n";
    q.Enqueue(3);
    std::cout<< "front: " << q.front() <<" back: "<< q.back()<< " size:
" << q.Size() << "\n";
    q.Enqueue(4);
    std::cout<< "front: " << q.front() <<" back: "<< q.back()<< " size:
" << q.Size() << "\n";
    q.Enqueue(5);
    std::cout<< "front: " << q.front() <<" back: "<< q.back()<< " size:
" << q.Size() << "\n";
```

```
    q.Enqueue(6);
    std::cout<< "front: " << q.front() <<" back: "<< q.back()<< " size: " << q.Size() << "\n";

    return 0;
}
```



COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.